

```
In [24]: import pandas as pd
import numpy as np
import tensorflow as tf

print("OK")
```

OK

```
In [25]: data = pd.read_csv("data/Churn_Modelling.csv")
data.head()
```

Out[25]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure
0	1	15634602	Hargrave	619	France	Female	42	2
1	2	15647311	Hill	608	Spain	Female	41	1
2	3	15619304	Onio	502	France	Female	42	8
3	4	15701354	Boni	699	France	Female	39	1
4	5	15737888	Mitchell	850	Spain	Female	43	2

```
In [26]: # Display general information about dataset (data types, missing values)
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   RowNumber              10000 non-null  int64  
1   CustomerId             10000 non-null  int64  
2   Surname                 10000 non-null  str     
3   CreditScore             10000 non-null  int64  
4   Geography               10000 non-null  str     
5   Gender                  10000 non-null  str     
6   Age                     10000 non-null  int64  
7   Tenure                  10000 non-null  int64  
8   Balance                 10000 non-null  float64 
9   NumOfProducts          10000 non-null  int64  
10  HasCrCard               10000 non-null  int64  
11  IsActiveMember          10000 non-null  int64  
12  EstimatedSalary         10000 non-null  float64 
13  Exited                  10000 non-null  int64  
dtypes: float64(2), int64(9), str(3)
memory usage: 1.1 MB
```

```
In [27]: # Display statistical summary of numerical columns
data.describe()
```

Out[27]:

	RowNumber	CustomerId	CreditScore	Age	Tenure	Bal
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.00
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.88
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.40
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.00
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.00
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.54
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.24
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.09



```
In [28]: # Remove irrelevant columns that do not contribute to prediction
# These columns are identifiers and have no predictive value
data = data.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)
```

```
In [29]: # Convert categorical variables (text) into numerical values
from sklearn.preprocessing import LabelEncoder

le_geo = LabelEncoder()
le_gender = LabelEncoder()

data['Geography'] = le_geo.fit_transform(data['Geography'])
data['Gender'] = le_gender.fit_transform(data['Gender'])
```

```
In [30]: # Separate input features (X) and target variable (y)
# 'Exited' is the target (0 = stayed, 1 = left)
X = data.drop('Exited', axis=1)
y = data['Exited']
```

```
In [31]: #Split data into train/test sets
from sklearn.model_selection import train_test_split

# 70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=101
)
```

```
In [32]: #Scaling

from sklearn.preprocessing import StandardScaler

# Normalize feature values to improve neural network performance
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [33]: #Create model
```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

# Create a sequential neural network model
model = Sequential()

# Define input layer
model.add(Input(shape=(X_train.shape[1],)))

# Add hidden layers with ReLU activation
model.add(Dense(16, activation='relu'))
model.add(Dense(8, activation='relu'))

# Output layer with sigmoid activation for binary classification
model.add(Dense(1, activation='sigmoid'))

model.summary()

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 16)	176
dense_7 (Dense)	(None, 8)	136
dense_8 (Dense)	(None, 1)	9

◀  ▶

Total params: 321 (1.25 KB)

Trainable params: 321 (1.25 KB)

Non-trainable params: 0 (0.00 B)

```

In [35]: # Compile the model with optimizer, loss function and evaluation metric
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

```

```

In [36]: # Train the neural network on training data
# Validate the model on test data during training
history = model.fit(
    X_train,
    y_train,
    validation_data=(X_test, y_test),
    epochs=30,
    batch_size=32
)

```

Epoch 1/30
219/219 ————— 2s 3ms/step - accuracy: 0.7366 - loss: 0.5597 - val_
accuracy: 0.7993 - val_loss: 0.4552
Epoch 2/30
219/219 ————— 0s 2ms/step - accuracy: 0.8139 - loss: 0.4399 - val_
accuracy: 0.8083 - val_loss: 0.4160
Epoch 3/30
219/219 ————— 1s 2ms/step - accuracy: 0.8256 - loss: 0.4193 - val_
accuracy: 0.8270 - val_loss: 0.3989
Epoch 4/30
219/219 ————— 0s 2ms/step - accuracy: 0.8334 - loss: 0.4047 - val_
accuracy: 0.8367 - val_loss: 0.3836
Epoch 5/30
219/219 ————— 1s 2ms/step - accuracy: 0.8431 - loss: 0.3916 - val_
accuracy: 0.8490 - val_loss: 0.3718
Epoch 6/30
219/219 ————— 0s 2ms/step - accuracy: 0.8470 - loss: 0.3819 - val_
accuracy: 0.8507 - val_loss: 0.3627
Epoch 7/30
219/219 ————— 0s 2ms/step - accuracy: 0.8500 - loss: 0.3728 - val_
accuracy: 0.8557 - val_loss: 0.3552
Epoch 8/30
219/219 ————— 1s 2ms/step - accuracy: 0.8506 - loss: 0.3652 - val_
accuracy: 0.8583 - val_loss: 0.3497
Epoch 9/30
219/219 ————— 1s 2ms/step - accuracy: 0.8534 - loss: 0.3592 - val_
accuracy: 0.8607 - val_loss: 0.3449
Epoch 10/30
219/219 ————— 0s 2ms/step - accuracy: 0.8553 - loss: 0.3549 - val_
accuracy: 0.8613 - val_loss: 0.3418
Epoch 11/30
219/219 ————— 1s 2ms/step - accuracy: 0.8557 - loss: 0.3520 - val_
accuracy: 0.8593 - val_loss: 0.3405
Epoch 12/30
219/219 ————— 0s 2ms/step - accuracy: 0.8566 - loss: 0.3496 - val_
accuracy: 0.8597 - val_loss: 0.3393
Epoch 13/30
219/219 ————— 0s 2ms/step - accuracy: 0.8564 - loss: 0.3478 - val_
accuracy: 0.8620 - val_loss: 0.3383
Epoch 14/30
219/219 ————— 0s 2ms/step - accuracy: 0.8570 - loss: 0.3455 - val_
accuracy: 0.8627 - val_loss: 0.3398
Epoch 15/30
219/219 ————— 1s 2ms/step - accuracy: 0.8573 - loss: 0.3454 - val_
accuracy: 0.8637 - val_loss: 0.3366
Epoch 16/30
219/219 ————— 0s 2ms/step - accuracy: 0.8573 - loss: 0.3439 - val_
accuracy: 0.8587 - val_loss: 0.3373
Epoch 17/30
219/219 ————— 1s 2ms/step - accuracy: 0.8571 - loss: 0.3434 - val_
accuracy: 0.8637 - val_loss: 0.3360
Epoch 18/30
219/219 ————— 1s 2ms/step - accuracy: 0.8564 - loss: 0.3423 - val_
accuracy: 0.8623 - val_loss: 0.3353
Epoch 19/30
219/219 ————— 1s 2ms/step - accuracy: 0.8586 - loss: 0.3414 - val_
accuracy: 0.8637 - val_loss: 0.3352
Epoch 20/30
219/219 ————— 1s 2ms/step - accuracy: 0.8584 - loss: 0.3404 - val_
accuracy: 0.8637 - val_loss: 0.3354

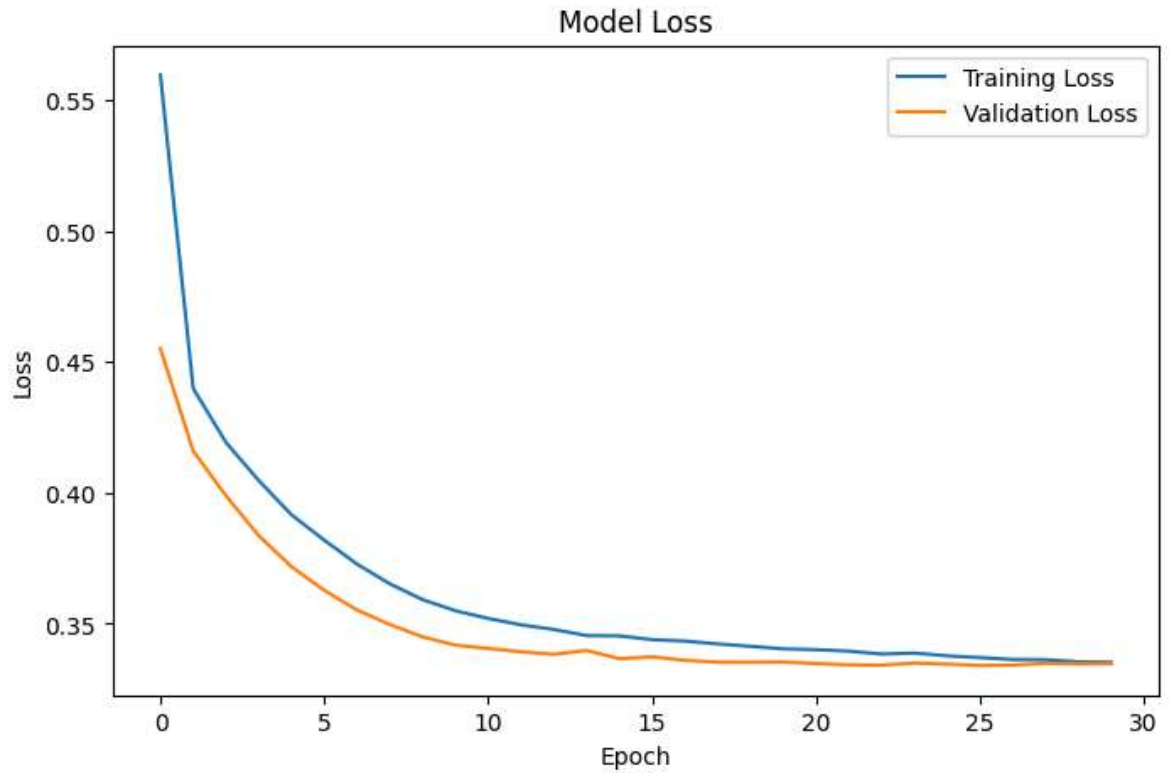
Epoch 21/30
219/219 ————— 0s 2ms/step - accuracy: 0.8573 - loss: 0.3401 - val_
accuracy: 0.8653 - val_loss: 0.3348
Epoch 22/30
219/219 ————— 0s 2ms/step - accuracy: 0.8574 - loss: 0.3395 - val_
accuracy: 0.8660 - val_loss: 0.3343
Epoch 23/30
219/219 ————— 0s 2ms/step - accuracy: 0.8584 - loss: 0.3384 - val_
accuracy: 0.8637 - val_loss: 0.3341
Epoch 24/30
219/219 ————— 1s 2ms/step - accuracy: 0.8564 - loss: 0.3388 - val_
accuracy: 0.8650 - val_loss: 0.3349
Epoch 25/30
219/219 ————— 1s 2ms/step - accuracy: 0.8589 - loss: 0.3377 - val_
accuracy: 0.8640 - val_loss: 0.3346
Epoch 26/30
219/219 ————— 0s 2ms/step - accuracy: 0.8594 - loss: 0.3370 - val_
accuracy: 0.8633 - val_loss: 0.3340
Epoch 27/30
219/219 ————— 0s 2ms/step - accuracy: 0.8601 - loss: 0.3363 - val_
accuracy: 0.8640 - val_loss: 0.3342
Epoch 28/30
219/219 ————— 0s 2ms/step - accuracy: 0.8603 - loss: 0.3363 - val_
accuracy: 0.8653 - val_loss: 0.3349
Epoch 29/30
219/219 ————— 0s 2ms/step - accuracy: 0.8604 - loss: 0.3354 - val_
accuracy: 0.8657 - val_loss: 0.3346
Epoch 30/30
219/219 ————— 0s 2ms/step - accuracy: 0.8601 - loss: 0.3353 - val_
accuracy: 0.8623 - val_loss: 0.3347

In [37]: `import matplotlib.pyplot as plt`

```
# Plot training and validation loss to monitor learning process
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Model Loss')
plt.legend()

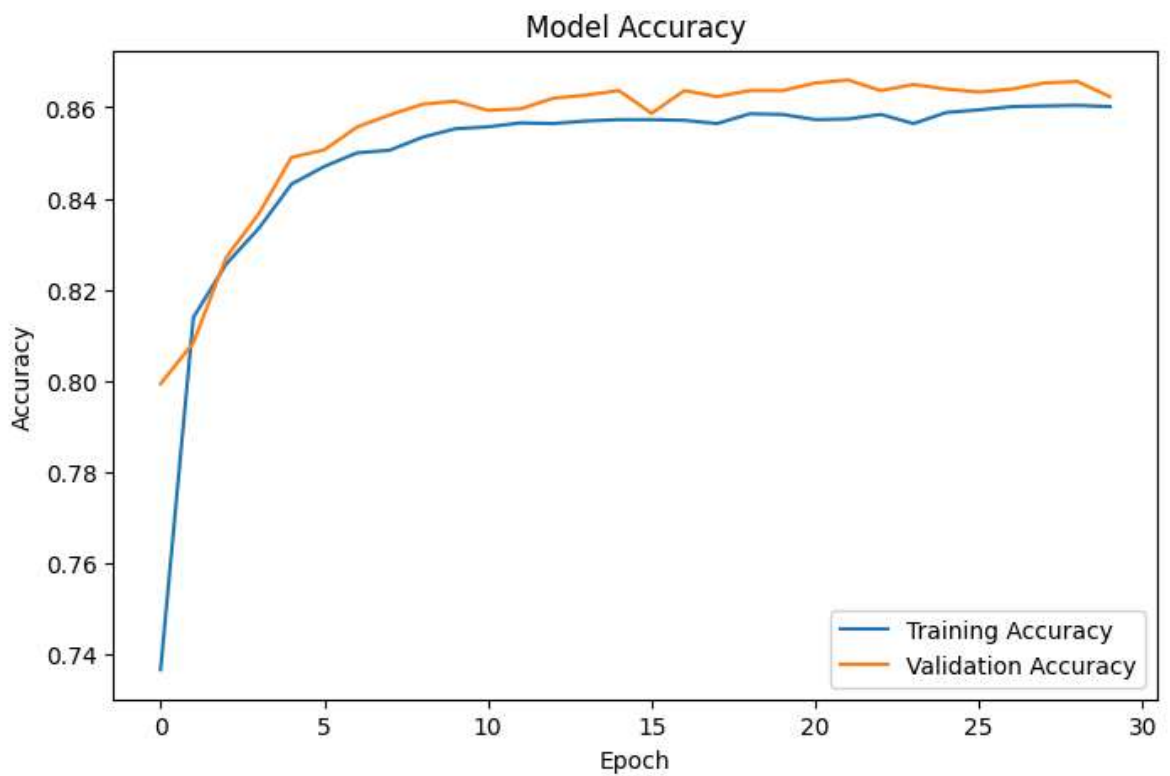
plt.show()
```



```
In [38]: # Plot training and validation accuracy
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Model Accuracy')
plt.legend()

plt.show()
```



```
In [39]: # Predict probabilities for test data
y_pred_prob = model.predict(X_test)

# Convert probabilities into binary predictions (threshold = 0.5)
y_pred = (y_pred_prob > 0.5).astype(int)
```

94/94 ————— 0s 2ms/step

```
In [40]: from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Calculate accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Display detailed classification metrics
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Accuracy: 0.8623333333333333

Confusion Matrix:

```
[[2311  67]
 [ 346 276]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.97	0.92	2378
1	0.80	0.44	0.57	622
accuracy			0.86	3000
macro avg	0.84	0.71	0.74	3000
weighted avg	0.86	0.86	0.85	3000

```
In [41]: # Conclusion:
# In this lab was built an Artificial Neural Network (ANN) for binary classifica
# The dataset was preprocessed by removing irrelevant columns and encoding categ
# The data was split into training and testing sets and scaled for better perfor
# A neural network model with Dense Layers was created and trained using the Ada
# The model was evaluated using accuracy, confusion matrix, and classification r
# The results demonstrate that the neural network can effectively predict custom
```